

LAB 2

Containerization & Kubernetes Orchestration

Task 2 — Manage Docker Containers and Images

Container lifecycle • Image tagging • Registry login • Image push • Docker Compose

Student	Kunal Prajapati
Local image	lab2-docker-app:v1
Additional tag	lab2-docker-app:latest
Registry image	kunalprajapati1/lab2-docker-app:v1
Compose services	web + redis

This report documents the Docker container/image management activities completed for Task 2, using the supplied terminal screenshots as evidence.

1. Container Lifecycle Management

The task required listing, starting, stopping and removing Docker containers. The evidence shows the application container **lab2-container** being inspected with **docker ps**, stopped with **docker stop**, started again with **docker start**, and later removed with **docker rm**. The final **docker ps -a** output confirms the removed container no longer appears.

```
PS D:\Desktop\lab2-task1> docker stop lab2-container
lab2-container
PS D:\Desktop\lab2-task1> docker rm lab2-container
lab2-container
PS D:\Desktop\lab2-task1> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
PS D:\Desktop\lab2-task1>
```

Figure. Docker container lifecycle commands and resulting container status.

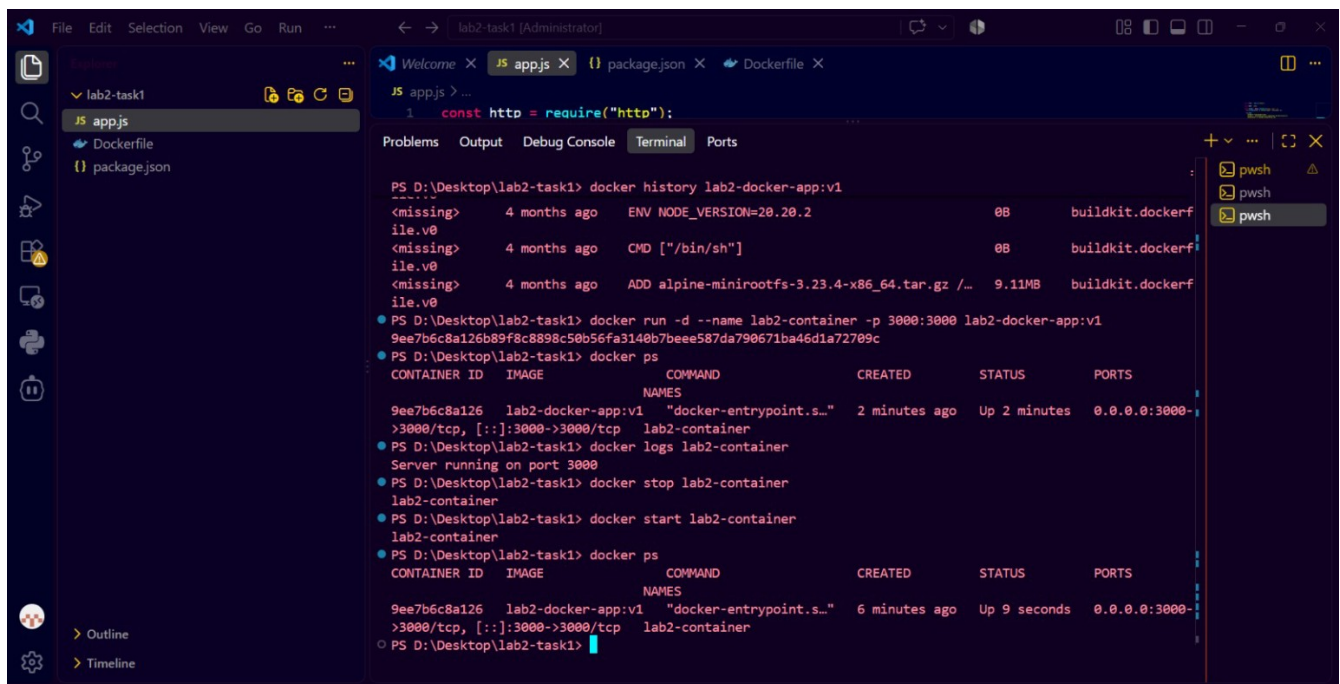


Figure. Docker container lifecycle commands and resulting container status.

Result

The container lifecycle was successfully demonstrated: the container could be stopped, restarted, and removed using Docker CLI commands.

2. Docker Image Tagging

The image **lab2-docker-app:v1** was given the additional **latest** tag using **docker tag lab2-docker-app:v1 lab2-docker-app:latest**. The image listing shows both tags referencing the same image ID, demonstrating that tags are references to the same underlying image rather than separate copies.

```
PS D:\Desktop\lab2-task1> docker compose ps
NAME                IMAGE                COMMAND              SERVICE  CREATED        STATUS          PORTS
lab2-compose-redis  redis:7-alpine      "docker-entrypoint.s...  redis    42 seconds ago  Up 41 seconds  6379/
tcp
lab2-compose-web    lab2-docker-app:v1  "docker-entrypoint.s...  web      42 seconds ago  Up 40 seconds  0.0.0
.0:3000->3000/tcp, [::]:3000->3000/tcp
PS D:\Desktop\lab2-task1>
```

Figure. Image tagged as both lab2-docker-app:v1 and lab2-docker-app:latest with the same image ID.

3. Docker Registry Login, Tagging and Push

Docker registry authentication was completed successfully. The local image was then tagged with the registry namespace **kunalprajapati1** as **kunalprajapati1/lab2-docker-app:v1**.

```
PS D:\Desktop\lab2-task1> docker login
Authenticating with existing credentials... [Username: kunalprajapati1]

Info → To Login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS D:\Desktop\lab2-task1> docker tag lab2-docker-app:v1 kunalprajapati1/lab2-docker-app:v1
PS D:\Desktop\lab2-task1> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
kunalprajapati1/lab2-docker-app:v1	3e16a9034e4a	193MB	48.4MB	
lab2-docker-app:latest	3e16a9034e4a	193MB	48.4MB	
lab2-docker-app:v1	3e16a9034e4a	193MB	48.4MB	

```
PS D:\Desktop\lab2-task1>
```

Figure. Successful Docker login and registry-qualified image tag.

The image was pushed using **docker push kunalprajapati1/lab2-docker-app:v1**. The terminal shows the image layers being pushed and the final v1 digest, confirming a successful registry upload.

```
PS D:\Desktop\lab2-task1> docker push kunalprajapati1/lab2-docker-app:v1
The push refers to repository [docker.io/kunalprajapati1/lab2-docker-app]
f8da4a911a58: Pushed
44136fa355b3: Pushed
4feea04c1543: Pushed
6a0ac1617861: Pushed
b2cbbfe903b0: Pushed
fff4e2c1b189: Pushed
a4844149f63e: Pushed
e44968c2c641: Pushed
82027f1f4aaa: Pushed
v1: digest: sha256:3e16a9034e4a296dc57d10e09436126eed31e801a24c09bf8da763e00ab34d5e size: 856
PS D:\Desktop\lab2-task1>
```

Figure. Successful docker push to the registry repository.

4. Docker Compose — Multi-Container Setup

Docker Compose was used to run a multi-container setup consisting of the web application and a Redis service. The Compose configuration uses **lab2-docker-app:v1** for the web service and **redis:7-alpine** for Redis.

```
PS D:\Desktop\lab2-task1> docker compose up -d
[+] up 2/2
  ✓ Network lab2-task1_default Created          0.0s
  ✓ Container lab2-compose-web Started          0.6s
PS D:\Desktop\lab2-task1> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
lab2-compose-web    lab2-docker-app:v1  "docker-entrypoint.s...  web        6 seconds ago  Up 5 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS D:\Desktop\lab2-task1>
```

Figure. Docker Compose startup showing the application and Redis containers being started.

```
PS D:\Desktop\lab2-task1> docker tag lab2-docker-app:v1 lab2-docker-app:latest
PS D:\Desktop\lab2-task1> docker images
Info → U In Use

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
lab2-docker-app:latest 3e16a9034e4a      193MB         48.4MB
lab2-docker-app:v1     3e16a9034e4a      193MB         48.4MB
PS D:\Desktop\lab2-task1>
```

Figure. docker compose ps showing the running web and Redis services.

Compose result

The Compose network was created and both services started successfully. The web service exposes port 3000, while Redis runs on its default container port 6379.

5. Verification Summary

The supplied evidence collectively demonstrates all required Task 2 operations: container listing and lifecycle management, image listing and tagging, registry authentication and image push, and multi-container orchestration using Docker Compose.

```
✓ Network lab2-task1_default Removed 0.6s
PS D:\Desktop\lab2-task1> docker compose up -d
[+] up 13/13
✓ Image redis:7-alpine Pulled 12.0s
✓ Network lab2-task1_default Created 0.1s
✓ Container lab2-compose-web Started 1.1s
✓ Container lab2-compose-redis Started 0.7s
PS D:\Desktop\lab2-task1> docker compose ps
```

Figure. Additional Task 2 execution evidence from the terminal.

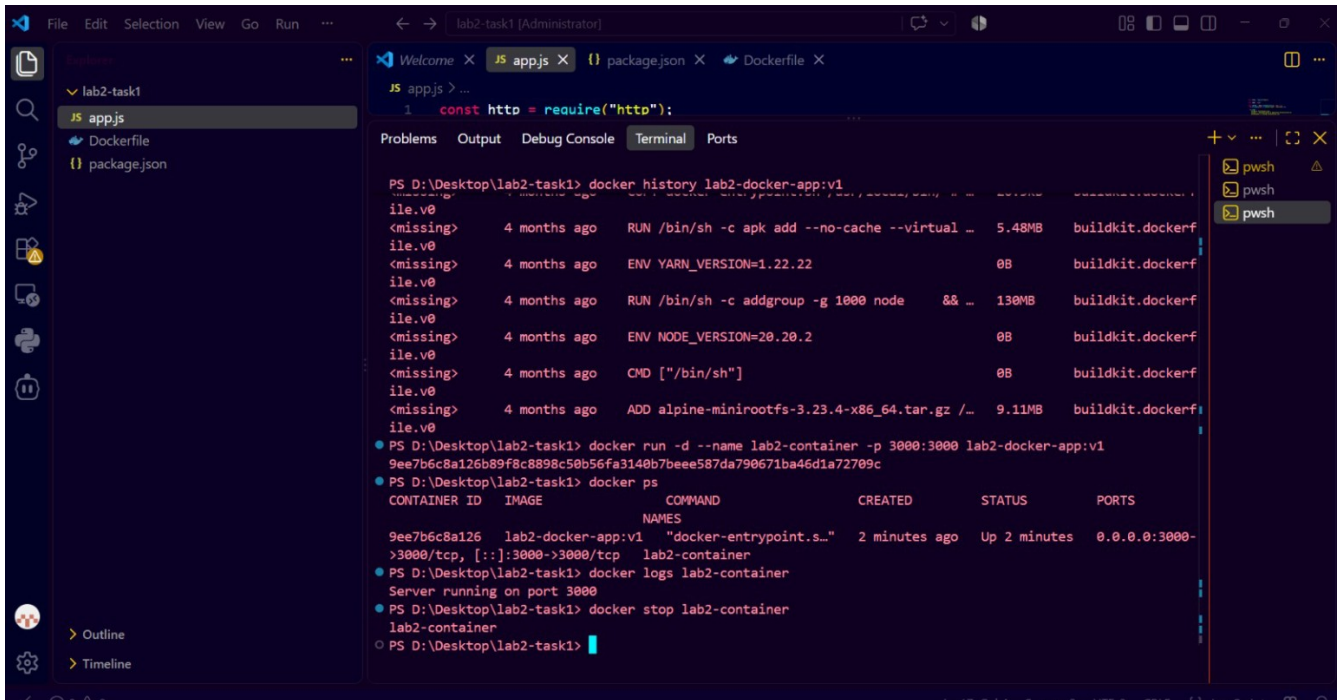


Figure. Additional Task 2 execution evidence from the terminal.

Requirement	Evidence / Result	Status
List containers	docker ps / docker ps -a outputs were captured.	Completed
Start containers	docker start lab2-container successfully restarted the container.	Completed
Stop containers	docker stop lab2-container successfully stopped it.	Completed
Remove containers	docker rm lab2-container removed the container; docker ps -a was empty after.	Completed
Tag images	lab2-docker-app:v1 was tagged as latest and with the Docker Hub namespace.	Completed
Push image to registry	docker push completed and returned a v1 digest.	Completed
Docker Compose	web and redis services were started and verified with docker compose ps.	Completed

Conclusion

Task 2 was completed successfully. Docker containers were managed through their lifecycle, the application image was tagged and pushed to a registry, and Docker Compose was used to run a web service together with Redis.

Submission note: Upload this PDF for Task 2. Keep the Docker project files and Compose configuration in the GitHub repository as the source-code record for the lab.